Maven™

Apache Maven

- Apache Maven to narzędzie służące do automatyzacji budowy oprogramowania.

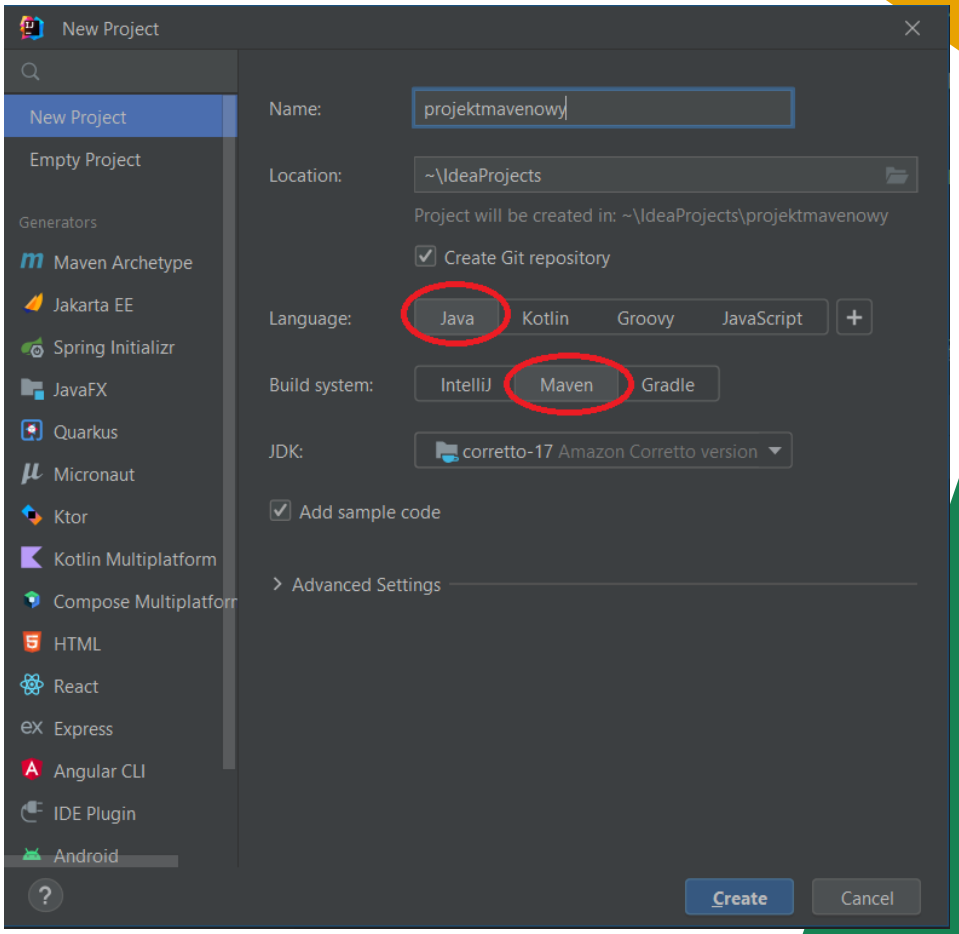
- Maven pomaga w zarządzaniu całym cyklem życia projektu, od kompilacji, przez testowanie, pakowanie, aż po wdrożenie i generowanie dokumentacji.

Apache Maven

- W IteliJ IDEA Maven jest wbudowany, więc w prosty sposób możemy utworzyć projekt mavenowy.
- Uruchamiamy IntelliJ IDEA i wybieramy opcję Create New Project na ekranie startowym lub File > New > Project...

Apache Maven

- W IntelIJ IDEA Maven jest wbudowany, więc w prosty sposób możemy utworzyć projekt mavenowy.
- Uruchamiamy IntelliJ IDEA i wybieramy opcję Create New Project na ekranie startowym lub File > New > Project...



Apache Maven

Project

- zadania2 C:\Users\kwasz\IdeaProjects\zadania2
  - .idea
  - src
    - main
      - java
        - org.example.wyklad
          - Hasher
          - Main
      - resources
    - test
      - java
  - target
  - .gitignore
  - password.txt
  - pom.xml
- External Libraries

Main.java | password.txt | pom.xml (zadania2) | Hasher.java

Cc W .* 3/3

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>zadania2</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
```

Apache Maven

- Plik **pom.xml** jest niezbędnym elementem projektu.
- Jest typu xml(eXtensible Markup Language)
- Zawiera konfigurację projektu oraz informacje o jego zależnościach, pluginach, wersjach i innych właściwościach.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>zadania2</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sou
    </properties>
```

pom.xml

- `xmlns="http://maven.apache.org/POM/4.0.0"`
  definiuje domyślną przestrzeń nazw XML dla dokumentu. Przestrzeń nazw jest mechanizmem XML, który dostarcza unikalny kontekst dla elementów i atrybutów.
- `xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
  deklaruje przestrzeń nazw XML Schema Instance. XML Schema jest językiem służącym do definiowania struktury i reguł walidacji dla dokumentów XML.
- `xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd"`
  Wskazuje lokalizację schematu XML Schema, który powinien być użyty do walidacji dokumentu. Podaje parserowi XML dokładną ścieżkę do pliku XSD (XML Schema Definition), który zawiera definicje typów danych, elementów i atrybutów dozwolonych w dokumencie, oraz reguły, które muszą być spełnione.

pom.xml

- Musimy też podać wersję modelu POM (Project Object Model):
  `<modelVersion>4.0.0</modelVersion>`
- Następnie mamy standardowe właściwości:
  `<groupId>org.example</groupId>`
  określa identyfikator grupy, do której należy projekt. Zazwyczaj jest to odwrócona nazwa domeny organizacji/developera.
  `<artifactId>projektmavenowy</artifactId>`
- Jest unikalną nazwą artefaktu generowanego przez projekt. Jest to nazwa, pod którą artefakt jest znany i przechowywany w repozytor
  `<version>1.0-SNAPSHOT</version>`

pom.xml

```
<groupId>org.example</groupId>
<artifactId>projektmavenowy</artifactId>
<version>1.0-SNAPSHOT</version>
```

Te 3 elemenety tworzą unikalny identyfikator artefaktu w systemie Mavena, który umożliwia zarządzanie zależnościami i ich wersjonowanie

pom.xml

```
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

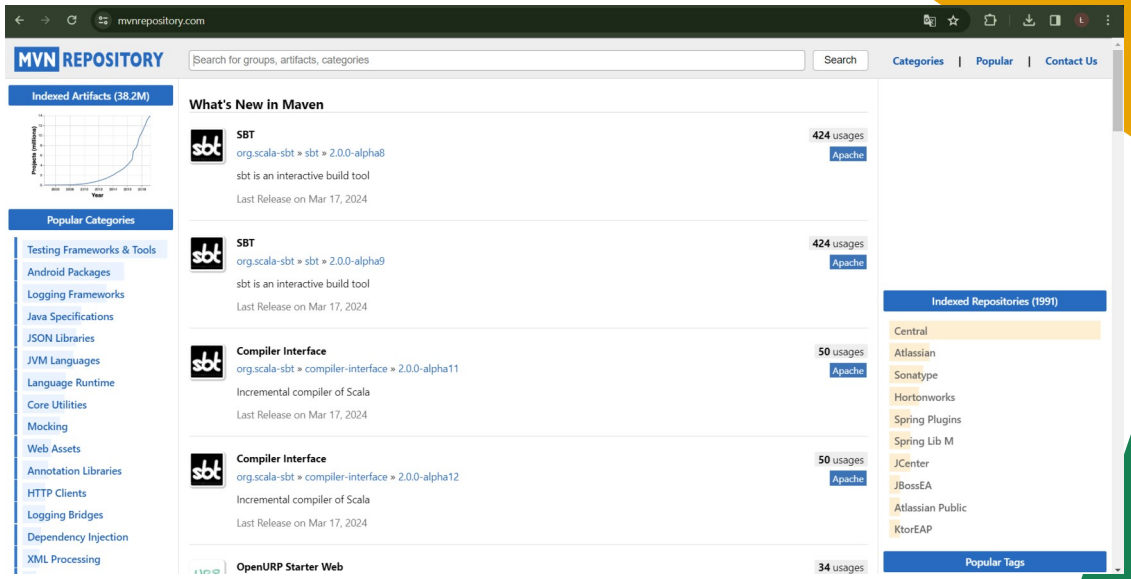
Ustawienia te kolejno opisują, jaką wersję javy używamy, z jaką wersją javy mają być kompatybilne skompilowane klasy, oraz jakie kodowanie ma być użyte dla plików źródłowych.

Zarządzanie zależnościami

Jest to kluczowa funkcjonalność Mavena ułatwia dodawanie, aktualizowanie i usuwanie zależności – automatyzując to zadanie .

Zarządzanie zależnościami

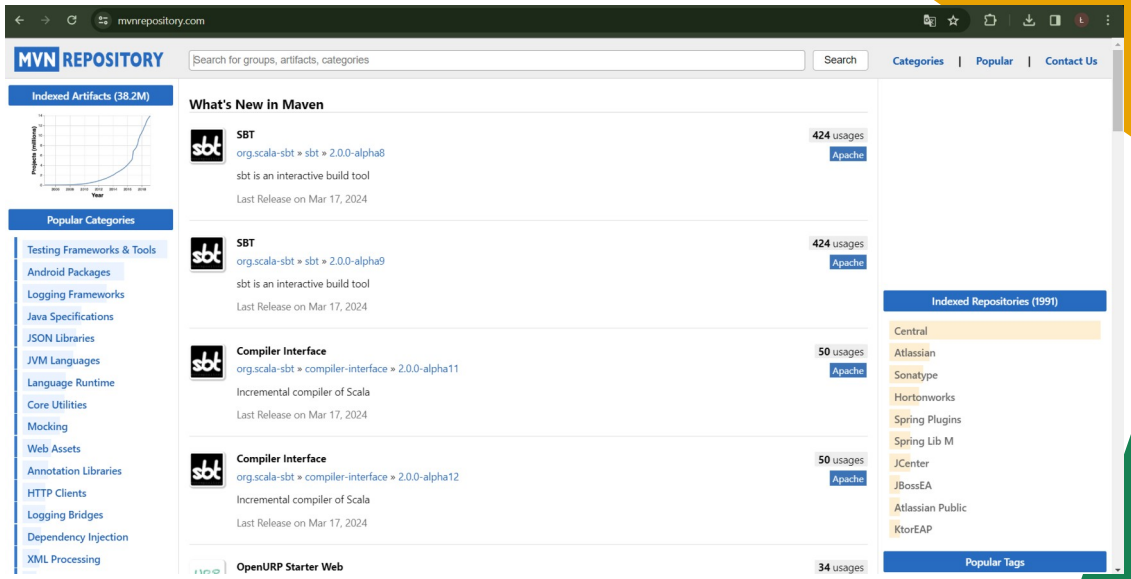
Maven korzysta z centralnych repozytoriów (takich jak Maven Central), gdzie przechowywane są artefakty (biblioteki, frameworki itp.), które mogą być używane jako zależności w projektach.



https://mvnrepository.com/

Zarządzanie zależnościami

Maven automatycznie rozwiązuje zależności co oznacza, że gdy dodajesz do projektu pewną zależność, Maven sam znajduje i pobiera wszystkie dodatkowe biblioteki, których ta zależność może wymagać do działania. Dzięki temu nie musisz ręcznie zarządzać każdą zależnością i jej zależnościami.



https://mvnrepository.com/

Zarządzanie zależnościami

Przykład: chcemy umożliwić aplikacji z ćwiczeń logowanie, dane użytkownika przechowujemy razem z hasłem podobnie jak informacje o pojazdach – w pliku.

Hasła nie mogą być przechowywane w pliku jako „plain-text”. Osoby nieuprawnione mogą mieć dostęp do haseł. W tym również my.

Hasła należy zahashować. A następnie porównywać zahashowane haslo podawane przy logowaniu z hashem z pliku.

Zarządzanie zależnościami

Zamiast samemu pisać metodę hashującą wykorzystamy rozwiązanie z repozytorium mavena:

https://mvnrepository.com/artifact/commons-codec/commons-codec/1.16.1

Zarządzanie zależnościami

Dodajemy zależność do pliku pom.xml,maven pobierze i doda ją za nas:

```
<dependencies>
    <dependency>
        <groupId>commons-codec</groupId>
        <artifactId>commons-codec</artifactId>
        <version>1.16.1</version>
    </dependency>
</dependencies>
```

Zarządzanie zależnościami

Teraz możemy użyć metodę służącą do hashowania importując odpowiednią klasę:

```
package org.example.wyklad;

import org.apache.commons.codec.digest.DigestUtils;

public class Hasher {
    public static String hashPassword(String password) {
        return DigestUtils.sha256Hex(password);
    }
}
```

Zarządzanie zależnościami

Przetestujmy działanie metody pobierając hasło od usera, hashując je, zapisując do pliku i porównując z hasłem podanym drugi raz (try,catch zostały pominięte):

```
package org.example.wyklad;

import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintWriter;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws FileNotFoundException {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your password: ");
        String hashed = Hasher.hashPassword(sc.nextLine());
        System.out.println("Hashed password: " + hashed);

        PrintWriter pw = new PrintWriter("password.txt");
        pw.println(hashed);
        pw.close();
        Scanner sc2 = new Scanner(new File("password.txt"));
        System.out.println("Enter your password: ");
        if (Hasher.hashPassword(sc.nextLine()).equals(sc2.nextLine())){
            System.out.println("VALID");
        }else{
            System.out.println("NOT VALID");
        }
    }
}
```

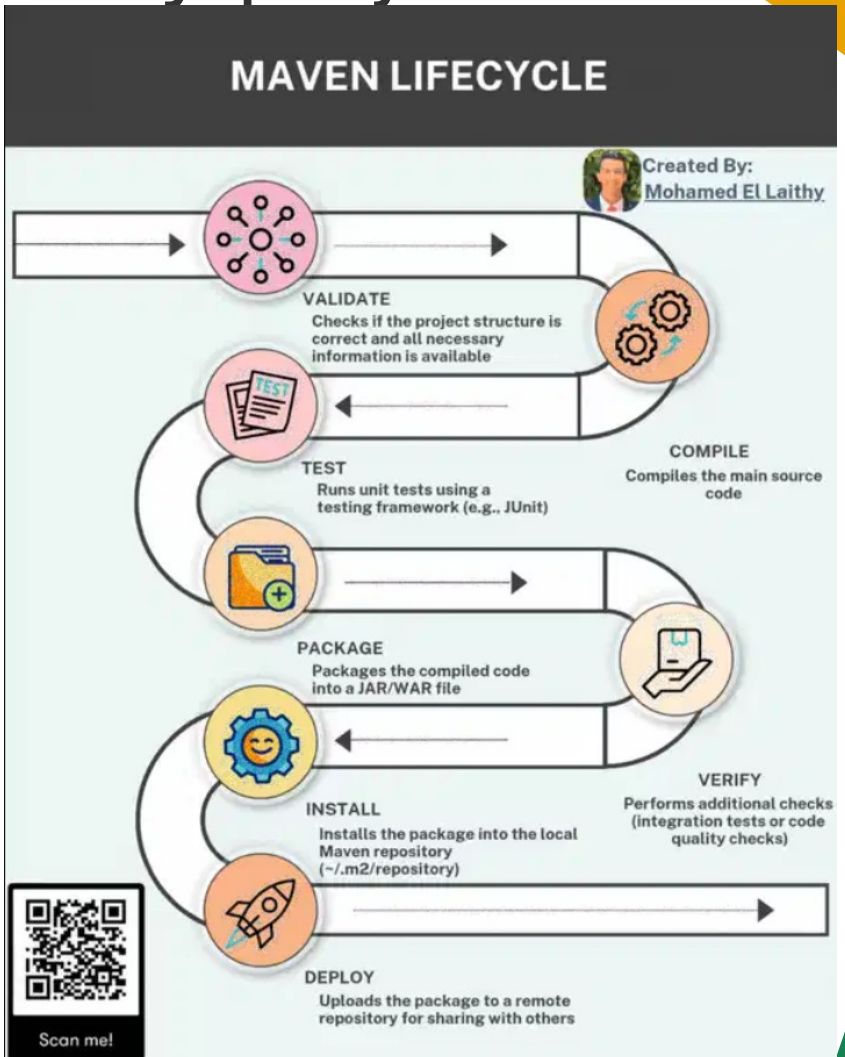
Zarządzanie zależnościami

Uruchomienie aplikacji:

```
C:\Users\kwasz\.jdks\corretto-17.0.18\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ
Enter your password:
abcd
Hashed password: 88d4266fd4e6338d13b845fcf289579d209c897823b9217da3e161936f031589
Enter your password:
abcd
VALID
```

Cykl życia budowy projektu

Cykl życia definiuje proces budowy i dystrybucji projektu. Składa się z uporządkowanych w kolejności faz, które należy wykonać.



Cykl życia budowy projektu

Istnieją 3 podstawowe cykle życia:

clean - czyści po sobie poprzednie efekty budowania aplikacji (często oznacza to wyczyszczenie folderu target)

site - buduje dokumentację projektu (np. javadoc)

domyślny cykl, który z kolei składa się z następujących faz (w prawidłowej kolejności):

validate: Sprawdza, czy projekt jest poprawny i czy wszystkie niezbędne informacje są dostępne.

compile: Kompiluje źródła projektu.

test: Testuje skompilowany kod źródłowy przy użyciu odpowiedniego frameworka testowego.

package: Pakuje skompilowany kod np. JAR w katalogu target.

verify: Sprawdza, czy pakiet jest poprawny i spełnia kryteria jakości.

install: Instaluje pakiet do lokalnego repozytorium, co umożliwia jego użycie jako zależności w innych projektach na tej samej maszynie.

deploy: Kopiuje ostateczny pakiet do zdalnego repozytorium, aby był udostępniony dla innych deweloperów i projektów.

Zbudowanie artefaktu i Instalacja

Okazuje się, że nasz artefakt przy próbie uruchomienia nie będzie działał. Z 2 powodów:
-brak pliku manifestu

-brak dołączonych zależności w paczce.

Pierwszy jak i drugi problem można rozwiązać za pomocą odpowiedniej wtyczki mavena użytej przy budowaniu projektu.

Artefakt - Paczka JAR = Java Archive

Stworzymy Fat Jar -czyli JAR-a, który zawiera:

Nasz kod,zależności,manifest

Zbudowanie artefaktu i Instalacja

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.5.2</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals>
                        <goal>shade</goal>
                    </goals>
                    <configuration>
                        <createDependencyReducedPom>false</createDependencyReducedPom>
                        <transformers>

                            <transformer implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                                <mainClass>org.example.wyklad.Main</mainClass>
                            </transformer>
                        </transformers>
                    </configuration>

                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

<build> — konfiguracja budowania projektu

<plugins> — lista pluginów używanych podczas builda

<plugin> — jeden konkretny plugin

maven-shade-plugin — plugin do tworzenia fat jara

<executions> — określa kiedy plugin ma się uruchomić

<phase>package</phase> — uruchom podczas package

<goal>shade</goal> — wykonaj operację shade

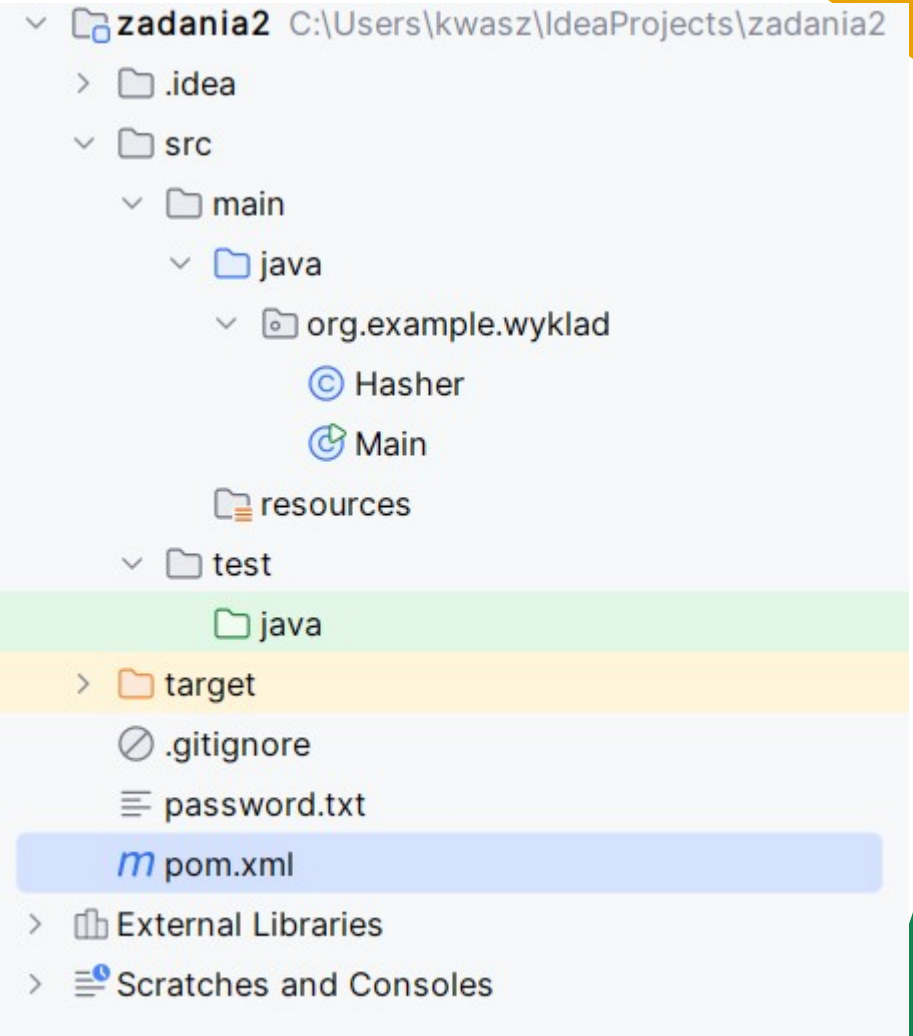
<configuration> — ustawienia pluginu

createDependencyReducedPom=false — nie twórz dodatkowego uproszczonego POM-a

ManifestResourceTransformer — zmodyfikuj manifest JAR-a

<mainClass>... — ustaw klasę startową aplikacji

Struktura katalogów

W projektach mavenowych obowiązuje ustalona struktura katalogów.

Kod źródłowy umieszczamy w :
src>main>java>paczki
Zasoby statyczne jak pliki konfiguracyjne, skrypty w:
src>main>resources
Testy w:
src>test>java